SotilityYieldEngine.sol

SotilityYieldEngine.sol

Refactor Goals:

Rename contract, logic, structs, and events

Use terms like:

Vault

AI Strategy

Cross-chain

Preserve your laddered-leverage staking structure

Future-proof for integrations with DeFi protocols (Aave, GMX, etc.)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

/**
 * @title SotilityYieldEngine
 * @notice AI-managed, cross-chain, laddered-leverage staking vault for stablecoins.
 */
contract SotilityYieldEngine is Ownable {
    struct Strategy {
        string name;
        address targetProtocol;
        uint256 allocationBps;  // Allocation in basis points (1% = 100)
        uint256 leverageRatio;  // Multiplier, e.g., 2 = 2x
        uint256 lockTime;       // Lock duration in seconds
    }

    IERC20 public stableToken;
    address public rewardDistributor;

    Strategy[] public strategies;

    mapping(address => uint256) public deposits;
    mapping(address => uint256) public entryTimestamps;

    event Deposited(address indexed user, uint256 amount);
    event Withdrawn(address indexed user, uint256 amount);
    event StrategyAdded(string name);

    constructor(address _stableToken, address _rewardDistributor) {
        stableToken = IERC20(_stableToken);
        rewardDistributor = _rewardDistributor;
    }

    function deposit(uint256 amount) external {
        require(amount > 0, "Amount must be greater than 0");

        stableToken.transferFrom(msg.sender, address(this), amount);
        deposits[msg.sender] += amount;
        entryTimestamps[msg.sender] = block.timestamp;
        emit Deposited(msg.sender, amount);
    }

    function withdraw(uint256 amount) external {
        require(deposits[msg.sender] >= amount, "Insufficient balance");
        deposits[msg.sender] -= amount;
        stableToken.transfer(msg.sender, amount);
        emit Withdrawn(msg.sender, amount);
    }

    function addStrategy(
        string calldata name,
        address targetProtocol,
        uint256 allocationBps,
        uint256 leverageRatio,
        uint256 lockTime
    ) external onlyOwner {
        strategies.push(Strategy({
            name: name,
            targetProtocol: targetProtocol,
            allocationBps: allocationBps,
            leverageRatio: leverageRatio,
            lockTime: lockTime
        }));
        emit StrategyAdded(name);
    }

    function rebalanceYieldStrategies() external onlyOwner {
        // Placeholder for AI strategy engine:
        // Fetch APYs, risk scores, reallocate across strategies
        // Could use Chainlink Functions, Gelato, or custom oracle
    }

    function getUserStakeDetails(address user) external view returns (uint256 staked, uint256 duration) {
        return (deposits[user], block.timestamp - entryTimestamps[user]);
    }

    function getStrategies() external view returns (Strategy[] memory) {
        return strategies;
    }

    function setRewardDistributor(address _distributor) external onlyOwner {
        rewardDistributor = _distributor;
    }
}
```

Notein
Scan the QR code to download Notein